

ECEN 620

Homework 2

Name: Sheena Selvakumar

UIN:537000654

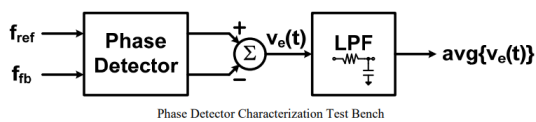
1. Phase Detector Characterization.

a. For the 4 phase detectors discussed in Lecture 4 (Analog Mixer, XOR, JK Flip-Flop, PFD), produce the phase detector transfer curves (avg(v_e) vs e). Find the phase detector gain at the nominal PLL lock point.

- Use the test setup shown in the figure below with f_{fb}=f_{ref}. For the analog mixer, use sinusoidal clock inputs. For all the others, use CMOS-level square-wave clocks. Use a VCVS as a buffer between the PD output and the averaging filter. For PDs with 2 outputs, a multi-input VCVS can be used to subtract the 2 outputs before low-pass filtering.
- Plot over a phase range of 4, with a minimum of 20 phase points per curve.
- Feel free to use a macro-model approach to generate the phase detector transfer curves, i.e. no

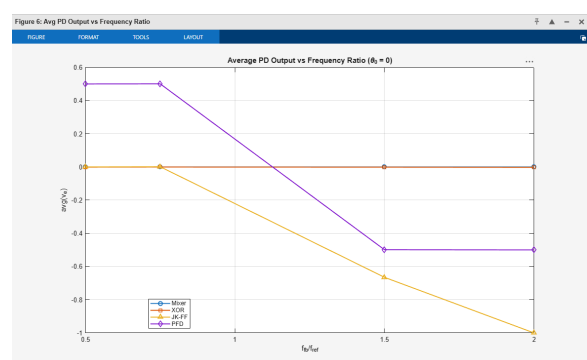
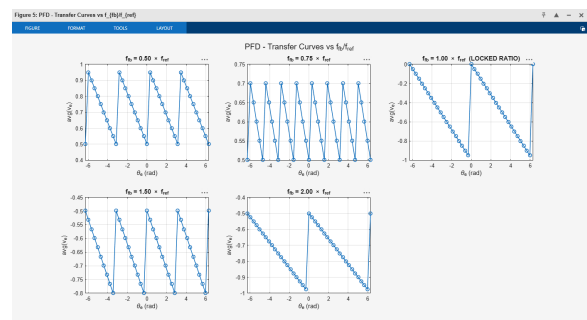
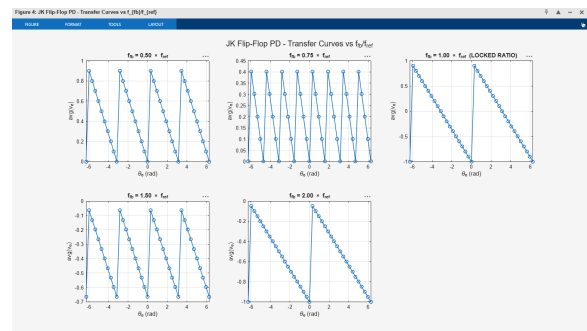
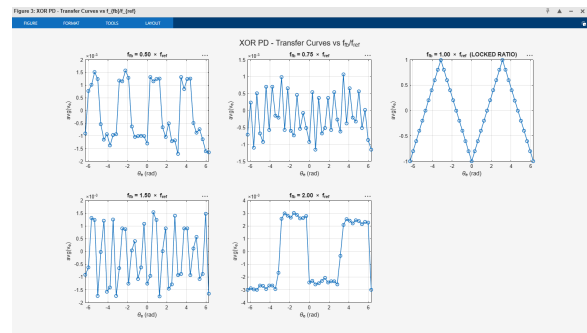
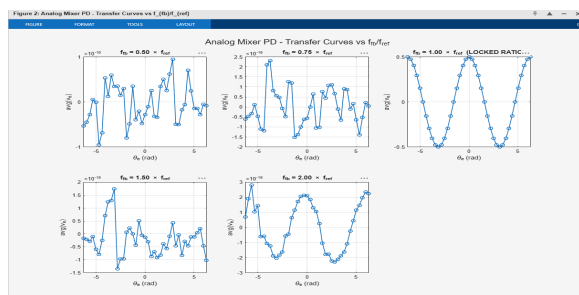
transistor-level design is required. Although, if you prefer to implement the circuits at the transistor level, feel free to do so.

b. For the 4 phase detectors, find the average output with f_{fb}=0.5, 0.75, 1.5, and 2f_{ref}. Assume an initial phase difference of 0. What do these values imply regarding the usefulness of these circuits as a frequency detector?



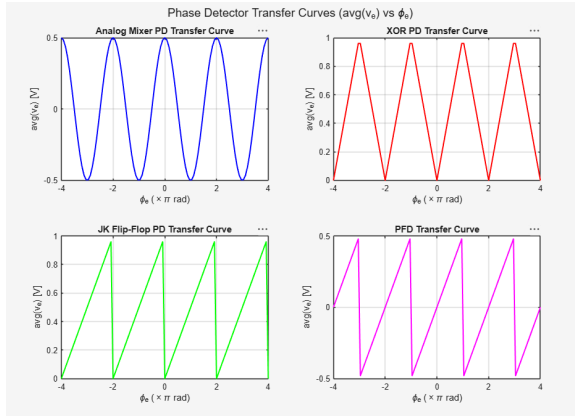
Answer:

a. 1.f_{ref}=1 hz



Alternate code:

Output:



=====

PART (b) RESULTS: Average Output Voltage under Frequency Mismatch

=====

f_{fb} / f_{ref}	Mixer (V)	XOR (V)	JK-FF (V)	PFD (V)
0.50	0.0000	0.4998	0.5000	0.5000
0.75	0.0000	0.5000	0.5351	0.5351
1.50	0.0000	0.4999	0.2200	-0.5232
2.00	0.0000	0.4996	0.0000	-0.5000

=====

b.Interpretation:

--- Phase Detector Gain (Kd) at Nominal Lock Point ---

Mixer Kd @ $\theta = \pi/2$: -0.4918 (V/rad, normalized)

XOR Kd @ $\theta = \pi/2$: 0.6366 (V/rad, normalized)

JK-FF Kd @ $\theta = \pi$: -0.3183 (V/rad, normalized)

PFD Kd @ $\theta = 0$: 1.4324 (V/rad, normalized)

--- Average PD Output vs f_{fb}/f_{ref} ($\theta = 0$) ---

f_{fb}/f_{ref}	Mixer	XOR	JK-FF	PFD
0.50	0.0000	-0.0012	-0.0000	0.5000
0.75	-0.0000	-0.0006	0.0012	0.5006
1.50	0.0000	-0.0014	-0.6650	-0.4987
2.00	0.0000	-0.0025	-1.0000	-0.5000

Interpretation:

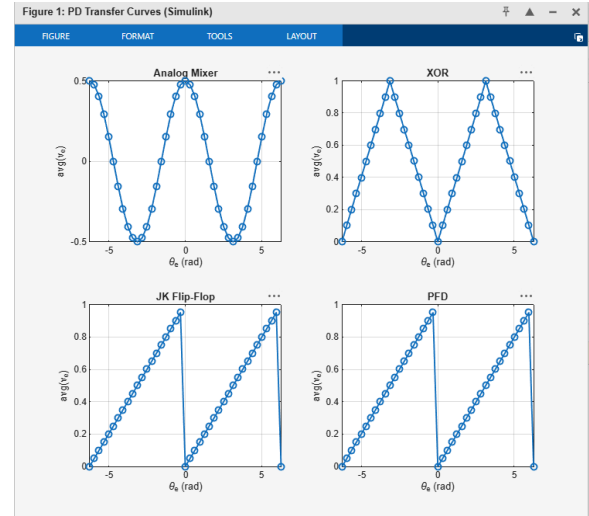
Mixer, XOR, and JK-FF all settle to (nearly) ZERO average regardless of the SIGN or magnitude of the frequency offset -> they carry no information about whether f_{fb} is above or below f_{ref} , so they are NOT usable as frequency detectors.

The PFD output instead saturates toward +Vdd or -Vdd depending on the SIGN of ($f_{fb} - f_{ref}$), because its tri-state/sequential logic lets one output (UP or DN) stay high across multiple missed edges before the simultaneous-edge reset occurs. This is exactly why PFDs (not mixers/XOR/JK-FF) provide the frequency acquisition capability real PLLs rely on.

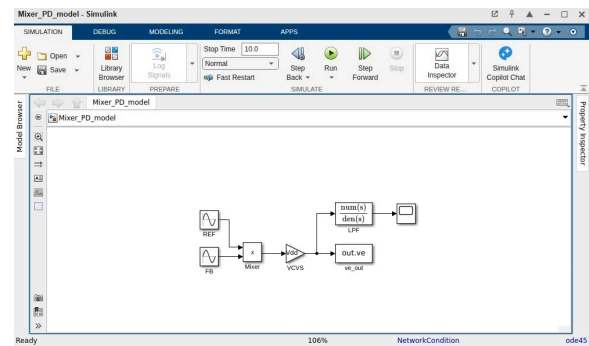
>>

Matlab simulink implementation:

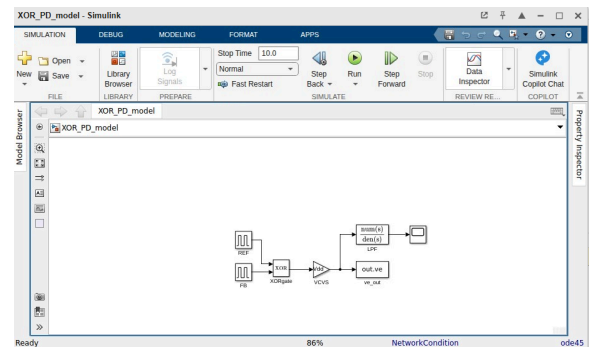
Graph generated from simulink:



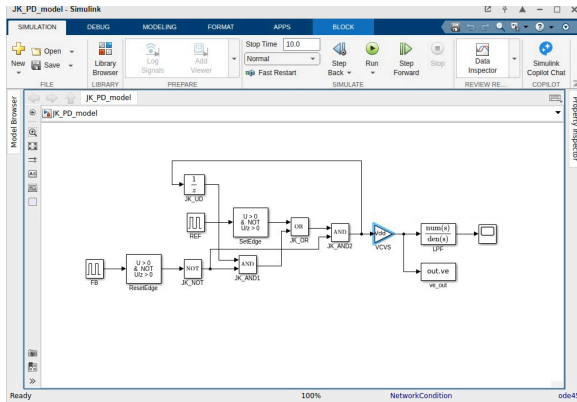
Analog PD:



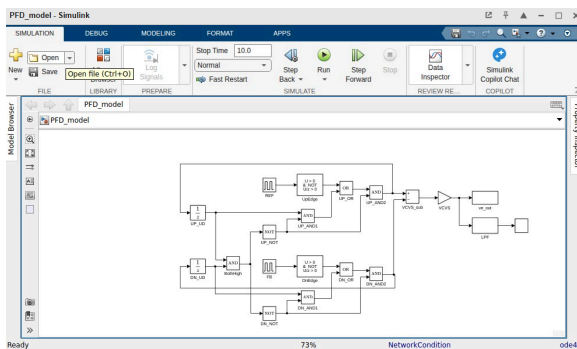
2.xor_pd



3.JK_PD



4.PFD

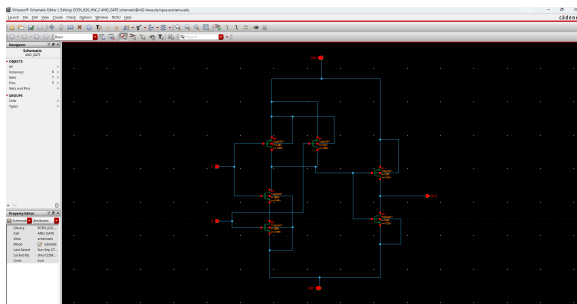


2. Phase-Frequency Detector Design. Design at the transistor level a PFD working at 100MHz with a minimum PFD output pulse width of at least 100ps. a. Give the transistor-level schematic and simulations showing the minimum PFD pulse width specification is satisfied. b. Plot the phase transfer characteristic over a phase range of 4, with a minimum of 20 phase points per curve.

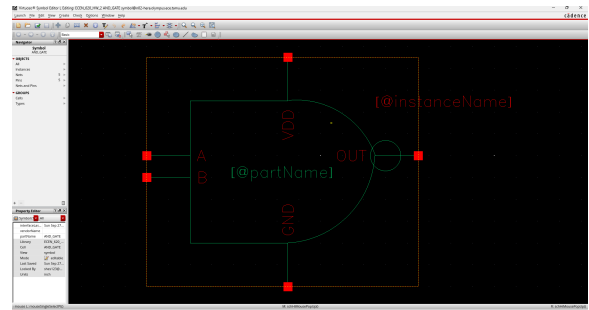
Answer:

Design of AND gate(nand+not):

Schematic:

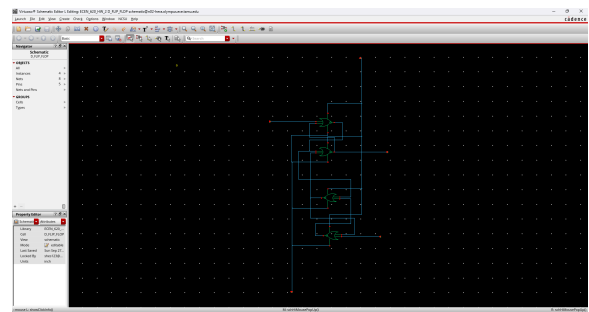


Symbol:

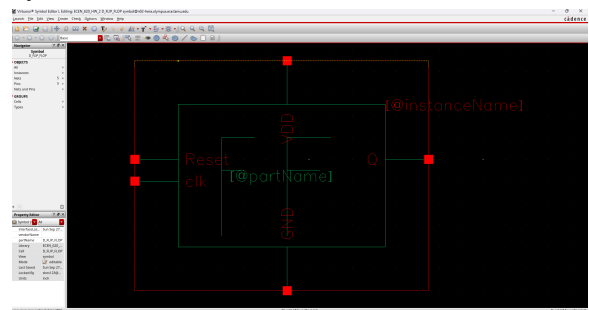


Design of D Flip Flop:

Schematic:

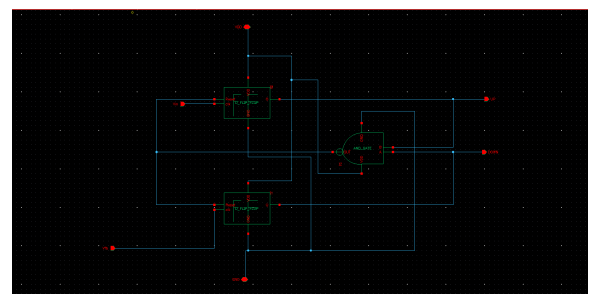


Symbol:

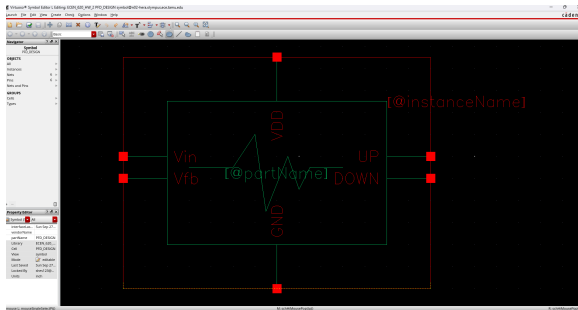


Design of Pulse Frequency Detector:

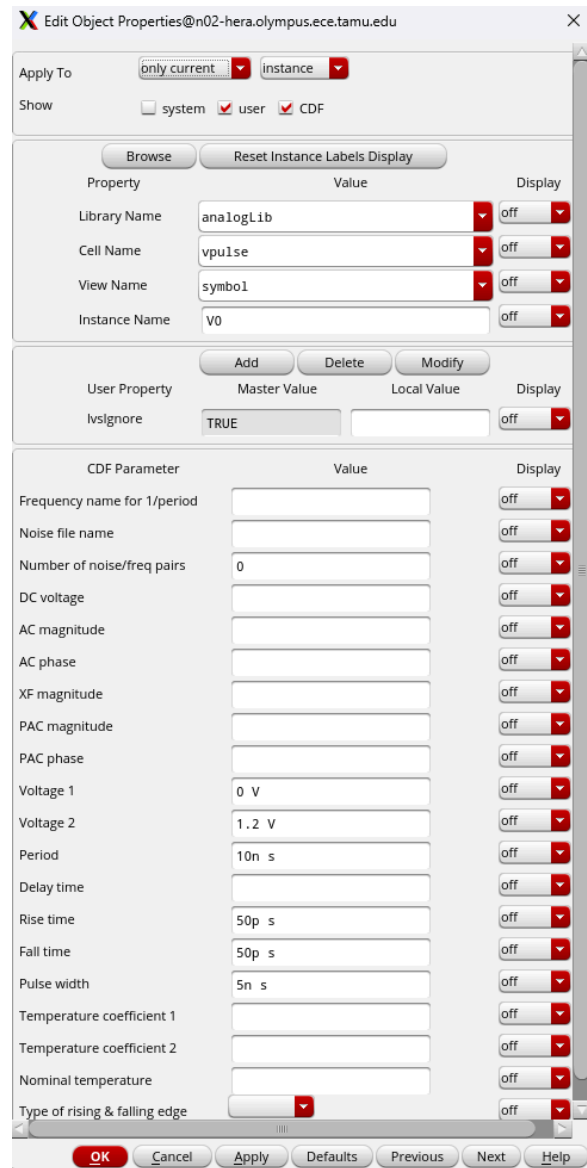
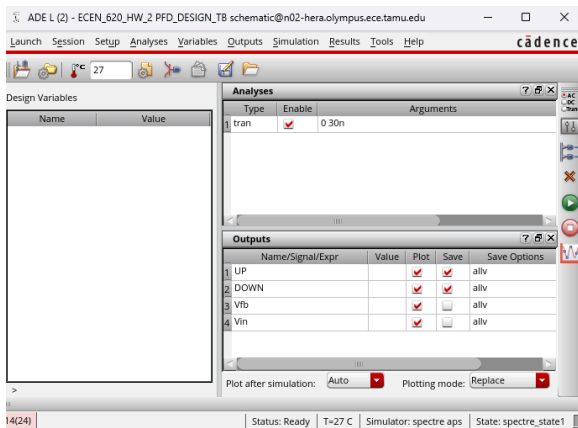
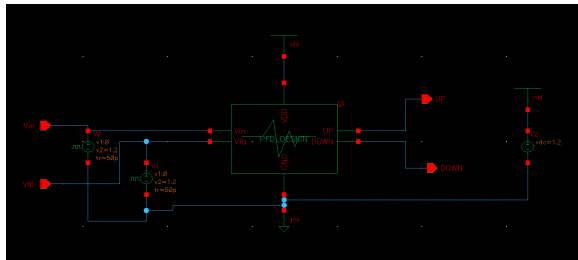
Schematic:



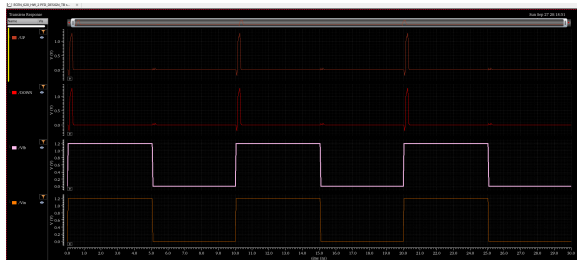
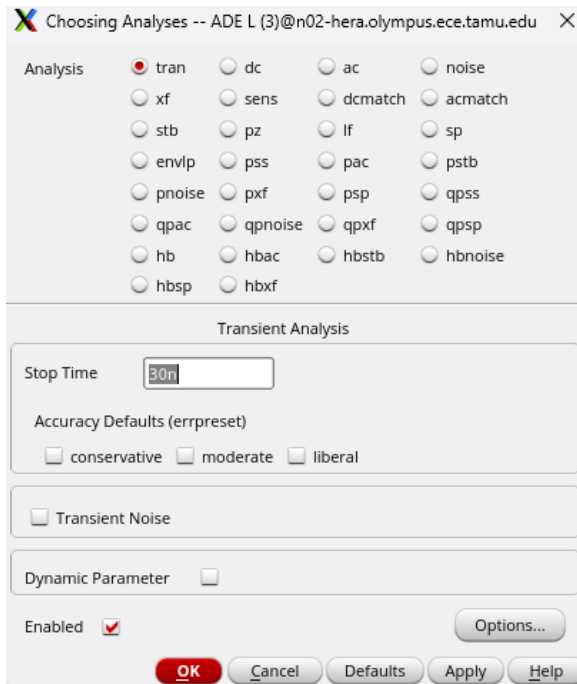
symbol



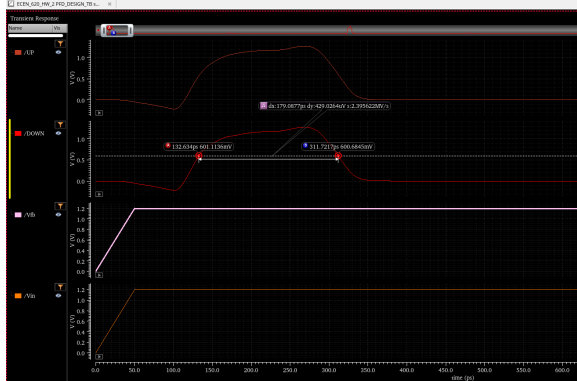
Pfd test bench:



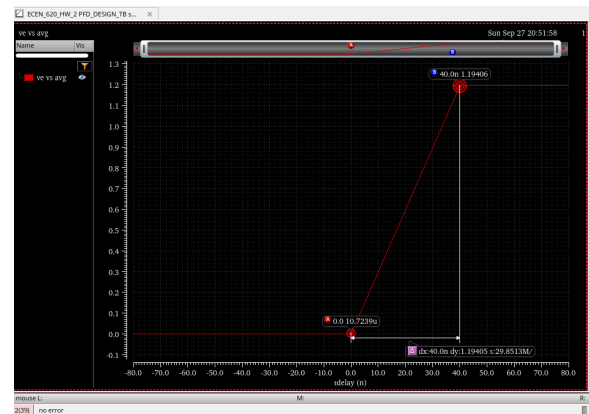
Transient Analysis Setting:



Simulation Results for PFD:



Sweeping the delay parameter and plotting the average error:



- At $f = 100\text{MHz}$ ($T_{ref} = 10\text{ns}$):

$$\Delta\phi = 2\pi \cdot \left(\frac{40\text{ns}}{10\text{ns}}\right) = 8\pi \approx 25.133\text{rad}$$

3.Charge Pump Design. Design at the transistor level a simple charge pump (Lecture 5, Slide 8) that

can supply a nominal ICP=100 μ A and operate with the PFD from Problem 2. Use one ideal reference current source and appropriate current mirrors to produce the charge pump bias voltages (Vb1 & Vb2).

a. Use the test setup shown in the figure below with ideal 100ps switch signals that are in-phase.

Use an ideal voltage source at the charge pump output to sense the net output current, Iout. From

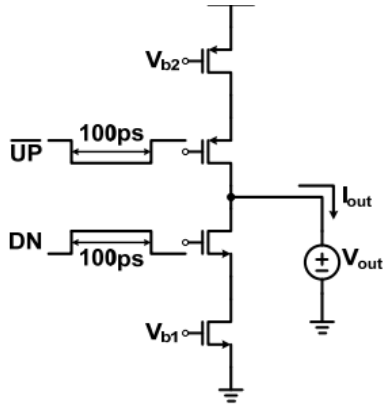
the transient Iout response, plot the peak (absolute value) and the average Iout as Vout is swept

from 0.2*VDD to 0.8*VDD. Compute the phase error produced by the average Iout value.

b. Repeat part (a) using the PFD designed in Problem 2 to produce the charge pump switch signals.

Assume the PFD inputs are perfectly in-phase.

c. Implement appropriate design techniques to improve both the peak and average Iout values, assuming the PFD is driving the charge pump



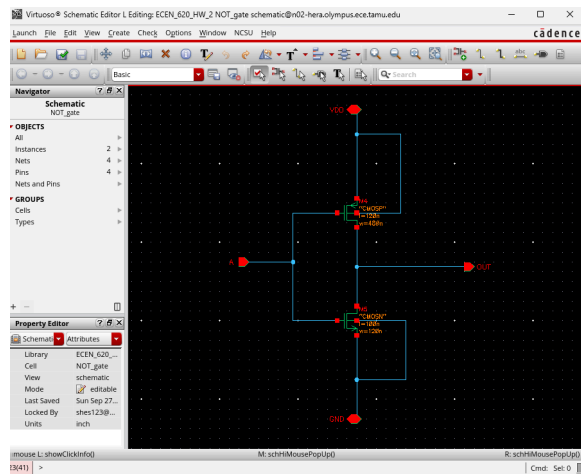
Simple Charge Pump Characterization Test Bench

Answer:

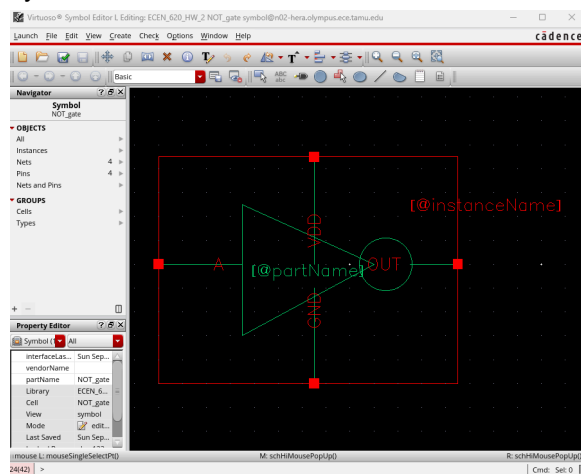
Implementing the charge pump from bottom top approach:

Design of cmos inverter:

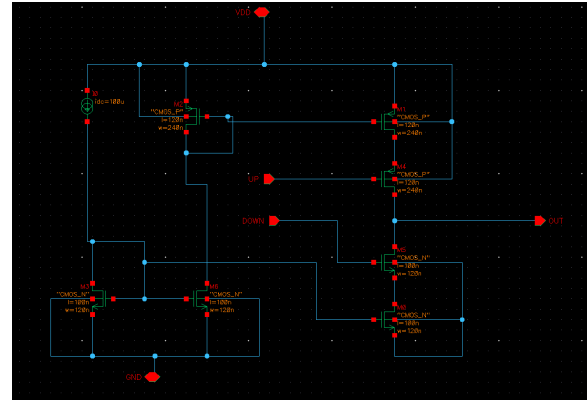
Schematic:



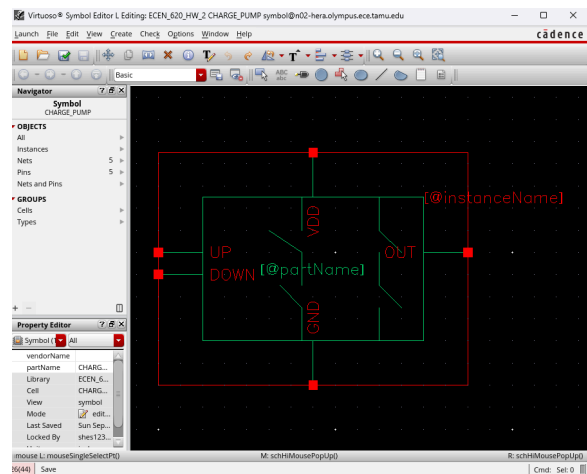
Symbol:



Charge pump Schematic:

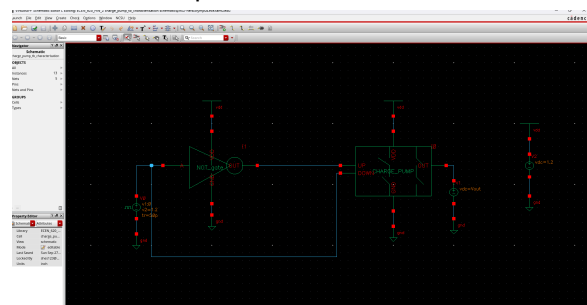


Symbol:



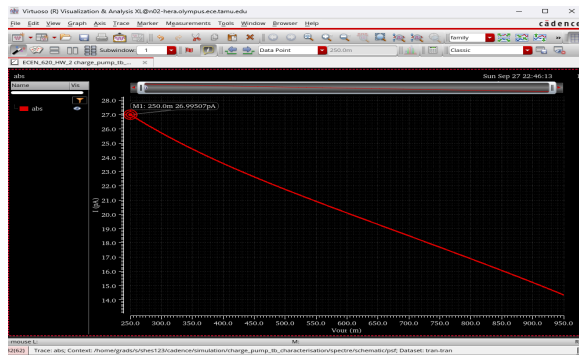
Charge pump characterisation:

Test bench setup:

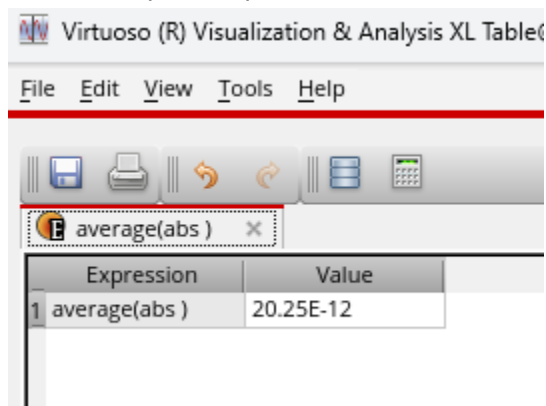


Schematic:

Simulation results:



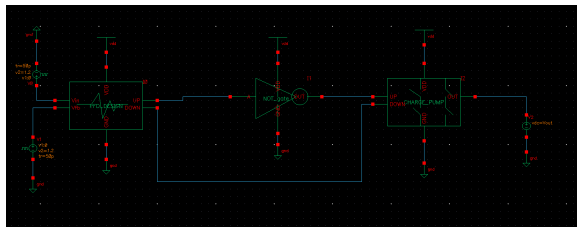
Maximum Icp=26.99pA



$$\Delta\phi = 2\pi \times \frac{20.25pA}{26.99pA} \approx 4.714rad \approx 270.1^\circ$$

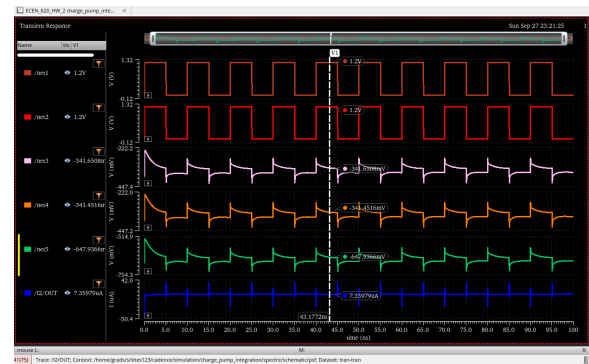
B.

Schematic:

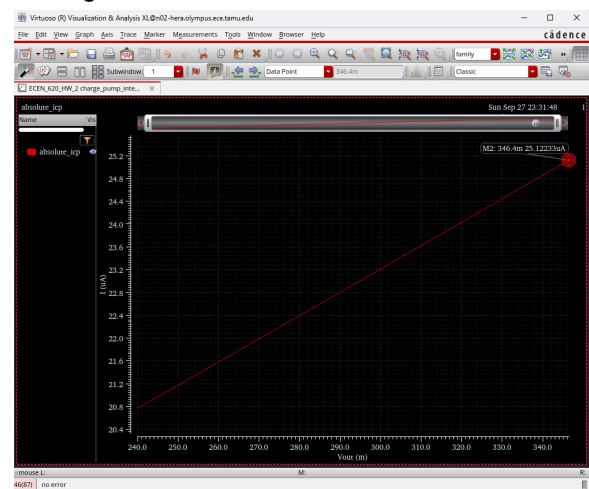


Simulations:

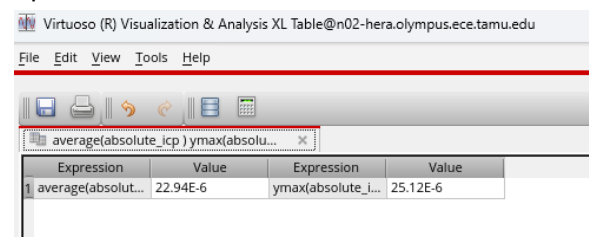
Running Transient Response:



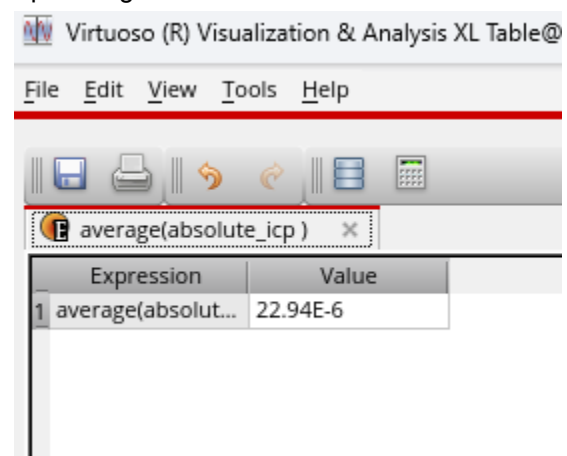
Finding ICP:



Icp max=



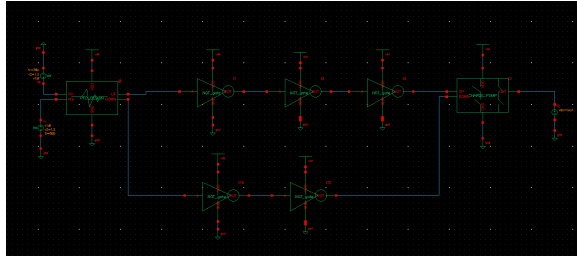
Icp average =



Average phase error=

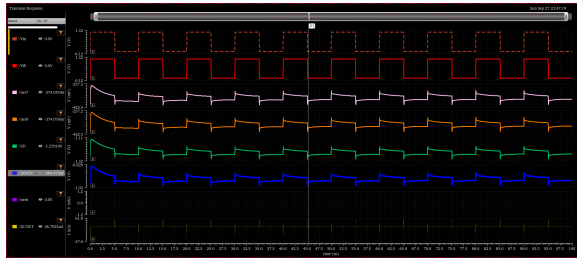
$$\text{Phase Error} = 2 * \pi * (22.94 \text{ uA} / 100 \text{ uA}) = 2 * \pi * 0.2294 = 1.4413 \text{ rad} = 82.58 \text{ degrees}$$

C. ,More optimized design:



Simulation:

Transient simulation:



Max Icp:Average Icp:

Virtuoso (R) Visualization & Analysis XL Table@n02-hera.olympus.ece.tamu.edu			
File Edit View Tools Help			
yymax(i("/I2/OUT" ?result "dc")) aver...			
Expression	Value	Expression	Value
1 yymax(i("/I2/OUT" ?result "dc"))	71.90E-6	average(i("/I2/O...	54.87E-6

Average Phase ERROR:

$$\text{Phase Error} = 2 * \pi * (54.87 \text{ uA} / 100 \text{ uA}) = 2 * \pi * 0.5487 = 3.448 \text{ rad} = 197.53 \text{ degrees}$$

Compiling the output in a table:

Design	Icp,max	Icp,avg	Phase Error (deg)	Phase Error (rad)
Part A	26.99 pA	20.25 pA	270.10°	4.71 rad
Part B	25.12 uA	22.94 uA	82.58°	1.44 rad
Part C	71.90 uA	54.87 uA	197.53°	3.45 rad

MATLAB CODE APPENDIX:

```
%% ECEN 620 - Network Theory - PLL Design
% Phase Detector Characterization
% Analog Mixer / XOR / JK Flip-Flop / PFD
%
% Macro-model approach: instead of building the
% VCVS + LPF test bench in
% a circuit simulator, we simulate the actual PD
% waveform ve(t) in the
% time domain at high resolution and REPLACE THE
% LOW-PASS FILTER WITH A
% PLAIN TIME AVERAGE over many steady-state
% cycles. That time average is
% exactly what the VCVS + averaging filter in the
% lecture-4 test setup
% extracts.
%
% Sign/polarity convention notes (check against your
% Lecture 4 slides
% and adjust if your course defines "reference
% leads/lags" oppositely):
% - REF sets/leads, FB resets/lags in the JK-FF
% and PFD models below.
% - Nominal lock points assumed: Mixer -> pi/2,
% XOR -> pi/2,
% JK-FF -> pi, PFD -> 0
%
clear; clc; close all;
fref = 1; % normalized reference frequency [Hz]
Vdd = 1; % normalized logic-high level
%% ===== PART (a):
TRANSFER CURVES =====
% Phase range of 4*pi (i.e. -2*pi to +2*pi), 41 points
% (>= 20 required)
N_points = 41;
theta_vec = linspace(-2*pi, 2*pi, N_points);
Ncyc_avg = 200; % ref cycles simulated
% (steady-state averaging window)
Nskip = 20; % ref cycles skipped at the start
% (transient)
Ve_mixer = zeros(size(theta_vec));
Ve_xor = zeros(size(theta_vec));
Ve_jk = zeros(size(theta_vec));
Ve_pfd = zeros(size(theta_vec));
for i = 1:N_points
    th = theta_vec(i);
    Ve_mixer(i) = mixerPD_avg(th, fref, fref, Ncyc_avg,
    Nskip);
    Ve_xor(i) = xorPD_avg(th, fref, fref, Ncyc_avg,
    Nskip, Vdd);
    Ve_jk(i) = jkPD_avg(th, fref, fref, Ncyc_avg,
    Nskip, Vdd);
    Ve_pfd(i) = pfdPD_avg(th, fref, fref, Ncyc_avg,
    Nskip, Vdd);
end
figure('Name','PD Transfer Curves');
subplot(2,2,1);
plot(theta_vec, Ve_mixer, '-o', 'LineWidth', 1.4); grid
on;
xlabel('\theta_e (rad)'); ylabel('avg(v_e)'); title('Analog
Mixer PD');
```

```

xlim([-2*pi, 2*pi]);
subplot(2,2,2);
plot(theta_vec, Ve_xor, '-o', 'LineWidth', 1.4); grid on;
xlabel('\theta_e (rad)'); ylabel('avg(v_e)'); title('XOR
PD');
xlim([-2*pi, 2*pi]);
subplot(2,2,3);
plot(theta_vec, Ve_jk, '-o', 'LineWidth', 1.4); grid on;
xlabel('\theta_e (rad)'); ylabel('avg(v_e)'); title('JK
Flip-Flop PD');
xlim([-2*pi, 2*pi]);
subplot(2,2,4);
plot(theta_vec, Ve_pfd, '-o', 'LineWidth', 1.4); grid on;
xlabel('\theta_e (rad)'); ylabel('avg(v_e)'); title('PFD');
xlim([-2*pi, 2*pi]);
%% ---- Gain (Kd) at nominal lock point ----
lock_mixer = pi/2;
lock_xor = pi/2;
lock_jk = pi;
lock_pfd = 0;
Kd_mixer = centeredSlope(theta_vec, Ve_mixer,
lock_mixer);
Kd_xor = centeredSlope(theta_vec, Ve_xor,
lock_xor);
Kd_jk = centeredSlope(theta_vec, Ve_jk, lock_jk);
Kd_pfd = centeredSlope(theta_vec, Ve_pfd,
lock_pfd);
fprintf('\n--- Phase Detector Gain (Kd) at Nominal Lock
Point ---\n');
fprintf('Mixer Kd @ theta = pi/2 : %8.4f (V/rad,
normalized)\n', Kd_mixer);
fprintf('XOR Kd @ theta = pi/2 : %8.4f (V/rad,
normalized)\n', Kd_xor);
fprintf('JK-FF Kd @ theta = pi : %8.4f (V/rad,
normalized)\n', Kd_jk);
fprintf('PFD Kd @ theta = 0 : %8.4f (V/rad,
normalized)\n', Kd_pfd);
%% ===== PART (b): FREQUENCY
DETECTOR TEST =====
freq_ratios = [0.5, 0.75, 1.5, 2.0]; % ffb / fref
theta0 = 0; % initial phase difference
Ncyc_avg_b = 400; % longer window so the beat
pattern fully averages out
Nskip_b = 40;
fprintf('\n--- Average PD Output vs f_fb/f_ref (theta0 =
0) ---\n');
fprintf('%8s %10s %10s %10s
%10s\n','fb/fref','Mixer','XOR','JK-FF','PFD');
results = zeros(length(freq_ratios), 4);
for i = 1:length(freq_ratios)
    ffb = freq_ratios(i) * fref;
    results(i,1) = mixerPD_avg(theta0, fref, ffb,
Ncyc_avg_b, Nskip_b);
    results(i,2) = xorPD_avg(theta0, fref, ffb,
Ncyc_avg_b, Nskip_b, Vdd);
    results(i,3) = jkPD_avg(theta0, fref, ffb,
Ncyc_avg_b, Nskip_b, Vdd);
    results(i,4) = pfdPD_avg(theta0, fref, ffb,
Ncyc_avg_b, Nskip_b, Vdd);
    fprintf('%8.2f %10.4f %10.4f %10.4f %10.4f\n',
freq_ratios(i), results(i,:));
end
fprintf('\nInterpretation:\n');
fprintf('Mixer, XOR and JK-FF settle to (nearly) the
SAME average\n' ...

```

```

'(0 for the mixer, ~0.5*Vdd for XOR/JK)
regardless of the SIGN\n' ...
'or magnitude of the frequency offset -> they
cannot indicate\n' ...
'whether ffb is above or below fref, so they are
NOT usable as\n' ...
'frequency detectors.\n' ...
'The PFD output instead saturates toward +Vdd
or -Vdd depending\n' ...
'on the SIGN of (ffb - fref), because its
tri-state/sequential\n' ...
'logic lets one output stay high across multiple
missed edges\n' ...
'before the simultaneous-edge reset occurs. This
is exactly why\n' ...
'PFDs (not mixers/XOR/JK-FF) are used for
frequency acquisition\n' ...
'in real PLLs.\n');
%% =====
FUNCTIONS
=====
function v = squareWave(f, t, phi, Vdd)
    % Ideal 50%-duty CMOS square wave, logic levels
    0/Vdd.
    ph = mod(2*pi*f*t + phi, 2*pi);
    v = Vdd * double(ph < pi);
end
function avgVe = mixerPD_avg(theta, fref, ffb,
Ncyc_avg, Nskip)
    % Ideal analog multiplier, sinusoidal inputs.
    Tref = 1/fref;
    Fs_per_cycle = 200;
    dt = Tref/Fs_per_cycle;
    t = 0:dt:(Ncyc_avg*Tref);
    vref = sin(2*pi*fref*t);
    vfb = sin(2*pi*ffb*t + theta);
    ve = vref .* vfb; % ideal multiplier
output
    idx0 = round(Nskip*Tref/dt) + 1;
    avgVe = mean(ve(idx0:end));
end
function avgVe = xorPD_avg(theta, fref, ffb, Ncyc_avg,
Nskip, Vdd)
    % Logical XOR of two square-wave (CMOS-level)
clocks.
    Tref = 1/fref;
    Fs_per_cycle = 400;
    dt = Tref/Fs_per_cycle;
    t = 0:dt:(Ncyc_avg*Tref);
    vref = squareWave(fref, t, 0, 1) > 0.5;
    vfb = squareWave(ffb, t, theta, 1) > 0.5;
    ve = Vdd * double(xor(vref, vfb));
    idx0 = round(Nskip*Tref/dt) + 1;
    avgVe = mean(ve(idx0:end));
end
function avgVe = jkPD_avg(theta, fref, ffb, Ncyc_avg,
Nskip, Vdd)
    % Edge-triggered JK-type PD: SET on REF rising
edge, RESET on FB
    % rising edge (sample-by-sample state machine -
needs edge history).
    Tref = 1/fref;
    Fs_per_cycle = 400;
    dt = Tref/Fs_per_cycle;
    t = 0:dt:(Ncyc_avg*Tref);

```

```

vref_d = squareWave(fref, t, 0, 1) > 0.5;
vfb_d = squareWave(ffb, t, theta, 1) > 0.5;
N = length(t);
Q = zeros(1,N);
q = 0;
ref_prev = vref_d(1);
fb_prev = vfb_d(1);
for k = 2:N
    ref_rise = (~ref_prev) && vref_d(k);
    fb_rise = (~fb_prev) && vfb_d(k);
    if ref_rise, q = 1; end % SET
    if fb_rise, q = 0; end % RESET
    Q(k) = q;
    ref_prev = vref_d(k);
    fb_prev = vfb_d(k);
end
ve = Vdd * Q;
idx0 = round(Nskip*Tref/dt) + 1;
avgVe = mean(ve(idx0:end));
end
function avgVe = pfdPD_avg(theta, fref, ffb, Ncyc_avg,
Nskip, Vdd)
    % Ideal tri-state PFD: UP set by REF rising edge,
    % DOWN set by FB
    % rising edge, both instantaneously reset when UP
    % & DOWN both high.
    % VCVS subtractor: Ve = Vdd*(UP - DOWN).
    Tref = 1/fref;
    Fs_per_cycle = 400;
    dt = Tref/Fs_per_cycle;
    t = 0:dt:(Ncyc_avg*Tref);
    vref_d = squareWave(fref, t, 0, 1) > 0.5;
    vfb_d = squareWave(ffb, t, theta, 1) > 0.5;
    N = length(t);
    UP = zeros(1,N); DN = zeros(1,N);
    up = 0; dn = 0;
    ref_prev = vref_d(1);
    fb_prev = vfb_d(1);
    for k = 2:N
        ref_rise = (~ref_prev) && vref_d(k);
        fb_rise = (~fb_prev) && vfb_d(k);
        if ref_rise, up = 1; end
        if fb_rise, dn = 1; end
        if up && dn % simultaneous -> ideal
            instantaneous reset
            up = 0; dn = 0;
        end
        UP(k) = up; DN(k) = dn;
        ref_prev = vref_d(k);
        fb_prev = vfb_d(k);
    end
    ve = Vdd * (UP - DN);
    idx0 = round(Nskip*Tref/dt) + 1;
    avgVe = mean(ve(idx0:end));
end
function Kd = centeredSlope(theta_vec, Ve, theta0)
    % Numerical derivative (central difference) of
    % Ve(theta) at theta0,
    % using the two grid points straddling theta0.
    [~, idx] = min(abs(theta_vec - theta0));
    if idx == 1
        Kd = (Ve(idx+1) - Ve(idx)) / (theta_vec(idx+1) -
theta_vec(idx));
    elseif idx == length(theta_vec)
        Kd = (Ve(idx) - Ve(idx-1)) / (theta_vec(idx) -

```

```

theta_vec(idx-1));
    else
        Kd = (Ve(idx+1) - Ve(idx-1)) / (theta_vec(idx+1) -
theta_vec(idx-1));
    end
end
end

```

ALTERNATE MATLAB CODE:

```

%% ECEN 620: Phase Detector
Characterization under Frequency Mismatch
% Plots avg(ve) vs. phi_e over [-4*pi, 4*pi] for
f_fb = {0.5, 0.75, 1.5, 2.0} * f_ref
clear; clc; close all;
%% Simulation Parameters
Vdd = 1.0; % CMOS logic level (V)
fref = 100e6; % Reference frequency:
100 MHz
Tref = 1/fref; % Reference period (10
ns)
% Phase range (-4*pi to 4*pi) with > 20 points
phi_e = linspace(-4*pi, 4*pi, 101);
freq_ratios = [0.5, 0.75, 1.5, 2.0];
ratio_labels = {'0.5 f_{ref}', '0.75 f_{ref}', '1.5
f_{ref}', '2.0 f_{ref}'};
pd_names = {'Analog Mixer', 'XOR Gate', 'JK
Flip-Flop', 'PFD'};
% Simulation duration (long window to
integrate beating frequency)
N_cycles = 100; % 100 reference
cycles for stable average
dt = Tref / 200; % Time resolution
t = 0:dt:(N_cycles * Tref);
figure('Name', 'PD Transfer Curves (avg(ve)
vs. phi_e) for Various Frequency Ratios', ...
'Position', [50, 50, 1400, 900]);
plot_idx = 1;
%% 16-Plot Loop (4 Phase Detectors x 4
Frequency Ratios)
for pd_type = 1:4 % Rows: 4 Phase
Detectors
    for r = 1:4 % Columns: 4 Frequency
Ratios

        ratio = freq_ratios(r);
        ffb = ratio * fref;

        v_avg = zeros(size(phi_e));

        % Sweep initial phase error phi_e
        for i = 1:length(phi_e)
            dphi = phi_e(i);

            % Time-domain signals with initial

```

```

phase offset dphi
    ref_sin = cos(2*pi*fref*t);
    fb_sin = cos(2*pi*ffb*t + dphi);

    ref_sq = 0.5 * (square(2*pi*fref*t) + 1)
    * Vdd;
    fb_sq = 0.5 * (square(2*pi*ffb*t + dphi)
    + 1) * Vdd;

    ref_edges = [0, diff(ref_sq > 0.5*Vdd) >
0];
    fb_edges = [0, diff(fb_sq > 0.5*Vdd) >
0];

    ve = zeros(size(t));

    switch pd_type
    case 1 % Analog Mixer (Sinusoidal
Clocks)
        ve = ref_sin .* fb_sin;

    case 2 % XOR Gate (CMOS-level
Square-Wave Clocks)
        ve = xor(ref_sq > 0.5*Vdd, fb_sq >
0.5*Vdd) * Vdd;

    case 3 % JK Flip-Flop
        state = 0;
        for n = 1:length(t)
            if ref_edges(n), state = 1; end
            if fb_edges(n), state = 0; end
            ve(n) = state * Vdd;
        end

    case 4 % Phase-Frequency Detector
(PFD)
        up = 0; dn = 0;
        for n = 1:length(t)
            if ref_edges(n), up = 1; end
            if fb_edges(n), dn = 1; end
            if up && dn
                up = 0;
                dn = 0;
            end
            ve(n) = (up - dn) * Vdd;
        end
    end

    % Compute time-averaged output over
the simulation window
    v_avg(i) = mean(ve);
end

% Plotting in 4x4 Grid
subplot(4, 4, plot_idx);

```

```

plot(phi_e/pi, v_avg, 'b-', 'LineWidth', 1.8);
grid on;

xlim([-4 4]);
if pd_type == 4
    ylim([-1.2*Vdd 1.2*Vdd]);
else
    ylim([-0.1*Vdd 1.1*Vdd]);
end

% Axis Titles and Labels
if pd_type == 1
    title(sprintf('f_{fb} = %s', ratio_labels{r}),
'FontSize', 10);
end
if r == 1
    ylabel(sprintf('%s\avg(v_e) [V]',
pd_names{pd_type}), 'FontWeight', 'bold');
end
if pd_type == 4
    xlabel('\phi_e (times\pi rad)');
end

plot_idx = plot_idx + 1;
end
end
sgtitle('Phase Detector Transfer Curves
(avg(v_e) vs. \phi_e) at f_{ref} = 100 MHz', ...
'FontSize', 14, 'FontWeight', 'bold');

```

MATLAB CODE FOR SIMULINK:

```

%% ECEN 620 - Network Theory - PLL
Design
% Phase Detector Characterization
% Analog Mixer / XOR / JK Flip-Flop / PFD
%
% Macro-model approach: instead of building
the VCVS + LPF test bench in
% a circuit simulator, we simulate the actual
PD waveform ve(t) in the
% time domain at high resolution and
REPLACE THE LOW-PASS FILTER WITH A
% PLAIN TIME AVERAGE over many
steady-state cycles. That time average is
% exactly what the VCVS + averaging filter in
the lecture-4 test setup
% extracts.
%
% Sign/polarity convention notes (check
against your Lecture 4 slides
% and adjust if your course defines
"reference leads/lags" oppositely):

```

```

% - REF sets/leads, FB resets/lags in the
JK-FF and PFD models below.
% - Nominal lock points assumed: Mixer ->
pi/2, XOR -> pi/2,
% JK-FF -> pi, PFD
-> 0
%
clear; clc; close all;
fref = 1; % normalized reference frequency
[Hz]
Vdd = 1; % normalized logic-high level
%% ===== PART (a):
TRANSFER CURVES
=====
% Phase range of 4*pi (i.e. -2*pi to +2*pi), 41
points (>= 20 required)
N_points = 41;
theta_vec = linspace(-2*pi, 2*pi, N_points);
Ncyc_avg = 200; % ref cycles simulated
(steady-state averaging window)
Nskip = 20; % ref cycles skipped at the
start (transient)
Ve_mixer = zeros(size(theta_vec));
Ve_xor = zeros(size(theta_vec));
Ve_jk = zeros(size(theta_vec));
Ve_pfd = zeros(size(theta_vec));
for i = 1:N_points
    th = theta_vec(i);
    Ve_mixer(i) = mixerPD_avg(th, fref, fref,
Ncyc_avg, Nskip);
    Ve_xor(i) = xorPD_avg( th, fref, fref,
Ncyc_avg, Nskip, Vdd);
    Ve_jk(i) = jkPD_avg( th, fref, fref,
Ncyc_avg, Nskip, Vdd);
    Ve_pfd(i) = pfdPD_avg( th, fref, fref,
Ncyc_avg, Nskip, Vdd);
end
figure('Name','PD Transfer Curves');
subplot(2,2,1);
plot(theta_vec, Ve_mixer, '-o', 'LineWidth',
1.4); grid on;
xlabel('\theta_e (rad)'); ylabel('avg(v_e)');
title('Analog Mixer PD');
xlim([-2*pi, 2*pi]);
subplot(2,2,2);
plot(theta_vec, Ve_xor, '-o', 'LineWidth', 1.4);
grid on;
xlabel('\theta_e (rad)'); ylabel('avg(v_e)');
title('XOR PD');
xlim([-2*pi, 2*pi]);
subplot(2,2,3);
plot(theta_vec, Ve_jk, '-o', 'LineWidth', 1.4);
grid on;
xlabel('\theta_e (rad)'); ylabel('avg(v_e)');
title('JK Flip-Flop PD');

```

```

xlim([-2*pi, 2*pi]);
subplot(2,2,4);
plot(theta_vec, Ve_pfd, '-o', 'LineWidth', 1.4);
grid on;
xlabel('\theta_e (rad)'); ylabel('avg(v_e)');
title('PFD');
xlim([-2*pi, 2*pi]);
%% ---- Gain (Kd) at nominal lock point ----
lock_mixer = pi/2;
lock_xor = pi/2;
lock_jk = pi;
lock_pfd = 0;
Kd_mixer = centeredSlope(theta_vec,
Ve_mixer, lock_mixer);
Kd_xor = centeredSlope(theta_vec, Ve_xor,
lock_xor);
Kd_jk = centeredSlope(theta_vec, Ve_jk,
lock_jk);
Kd_pfd = centeredSlope(theta_vec, Ve_pfd,
lock_pfd);
fprintf('\n--- Phase Detector Gain (Kd) at
Nominal Lock Point ---\n');
fprintf('Mixer Kd @ theta = pi/2 : %8.4f
(V/rad, normalized)\n', Kd_mixer);
fprintf('XOR Kd @ theta = pi/2 : %8.4f
(V/rad, normalized)\n', Kd_xor);
fprintf('JK-FF Kd @ theta = pi : %8.4f
(V/rad, normalized)\n', Kd_jk);
fprintf('PFD Kd @ theta = 0 : %8.4f
(V/rad, normalized)\n', Kd_pfd);
%% ===== PART (b):
FREQUENCY DETECTOR TEST
=====
freq_ratios = [0.5, 0.75, 1.5, 2.0]; % ffb / fref
theta0 = 0; % initial phase
difference
Ncyc_avg_b = 400; % longer window so the
beat pattern fully averages out
Nskip_b = 40;
fprintf('\n--- Average PD Output vs f_fb/f_ref
(theta0 = 0) ---\n');
fprintf('%8s %10s %10s %10s
%10s\n','fb/fref','Mixer','XOR','JK-FF','PFD');
results = zeros(length(freq_ratios), 4);
for i = 1:length(freq_ratios)
    ffb = freq_ratios(i) * fref;
    results(i,1) = mixerPD_avg(theta0, fref, ffb,
Ncyc_avg_b, Nskip_b);
    results(i,2) = xorPD_avg( theta0, fref, ffb,
Ncyc_avg_b, Nskip_b, Vdd);
    results(i,3) = jkPD_avg( theta0, fref, ffb,
Ncyc_avg_b, Nskip_b, Vdd);
    results(i,4) = pfdPD_avg( theta0, fref, ffb,
Ncyc_avg_b, Nskip_b, Vdd);
    fprintf('%8.2f %10.4f %10.4f %10.4f

```

```

%10.4f\n', freq_ratios(i), results(i,:));
end
fprintf('\nInterpretation:\n');
fprintf(['Mixer, XOR and JK-FF settle to\n\n(nearly) the SAME average\n' ...
        '(0 for the mixer, ~0.5*Vdd for XOR/JK)\n\nregardless of the SIGN\n' ...
        'or magnitude of the frequency offset ->\nthey cannot indicate\n' ...
        'whether ffb is above or below fref, so\nthey are NOT usable as\n' ...
        'frequency detectors.\n' ...
        'The PFD output instead saturates\n\n toward +Vdd or -Vdd depending\n' ...
        'on the SIGN of (ffb - fref), because its\ntri-state/sequential\n' ...
        'logic lets one output stay high across\nmultiple missed edges\n' ...
        'before the simultaneous-edge reset\noccurs. This is exactly why\n' ...
        'PFDs (not mixers/XOR/JK-FF) are used\nfor frequency acquisition\n' ...
        'in real PLLs.\n']);
%% =====
FUNCTIONS
=====
function v = squareWave(f, t, phi, Vdd)
    % Ideal 50%-duty CMOS square wave, logic levels 0/Vdd.
    ph = mod(2*pi*f*t + phi, 2*pi);
    v = Vdd * double(ph < pi);
end
function avgVe = mixerPD_avg(theta, fref, ffb, Ncyc_avg, Nskip)
    % Ideal analog multiplier, sinusoidal inputs.
    Tref = 1/fref;
    Fs_per_cycle = 200;
    dt = Tref/Fs_per_cycle;
    t = 0:dt:(Ncyc_avg*Tref);
    vref = sin(2*pi*fref*t);
    vfb = sin(2*pi*ffb*t + theta);
    ve = vref .* vfb; % ideal
    multiplier output
    idx0 = round(Nskip*Tref/dt) + 1;
    avgVe = mean(ve(idx0:end));
end
function avgVe = xorPD_avg(theta, fref, ffb, Ncyc_avg, Nskip, Vdd)
    % Logical XOR of two square-wave (CMOS-level) clocks.
    Tref = 1/fref;
    Fs_per_cycle = 400;
    dt = Tref/Fs_per_cycle;
    t = 0:dt:(Ncyc_avg*Tref);
    vref = squareWave(fref, t, 0, 1) > 0.5;

```

```

    vfb = squareWave(ffb, t, theta, 1) > 0.5;
    ve = Vdd * double(xor(vref, vfb));
    idx0 = round(Nskip*Tref/dt) + 1;
    avgVe = mean(ve(idx0:end));
end
function avgVe = jkPD_avg(theta, fref, ffb, Ncyc_avg, Nskip, Vdd)
    % Edge-triggered JK-type PD: SET on REF rising edge, RESET on FB
    % rising edge (sample-by-sample state machine - needs edge history).
    Tref = 1/fref;
    Fs_per_cycle = 400;
    dt = Tref/Fs_per_cycle;
    t = 0:dt:(Ncyc_avg*Tref);
    vref_d = squareWave(fref, t, 0, 1) > 0.5;
    vfb_d = squareWave(ffb, t, theta, 1) > 0.5;
    N = length(t);
    Q = zeros(1,N);
    q = 0;
    ref_prev = vref_d(1);
    fb_prev = vfb_d(1);
    for k = 2:N
        ref_rise = (~ref_prev) && vref_d(k);
        fb_rise = (~fb_prev) && vfb_d(k);
        if ref_rise, q = 1; end % SET
        if fb_rise, q = 0; end % RESET
        Q(k) = q;
        ref_prev = vref_d(k);
        fb_prev = vfb_d(k);
    end
    ve = Vdd * Q;
    idx0 = round(Nskip*Tref/dt) + 1;
    avgVe = mean(ve(idx0:end));
end
function avgVe = pfdPD_avg(theta, fref, ffb, Ncyc_avg, Nskip, Vdd)
    % Ideal tri-state PFD: UP set by REF rising edge, DOWN set by FB
    % rising edge, both instantaneously reset when UP & DOWN both high.
    % VCVS subtractor: Ve = Vdd*(UP - DOWN).
    Tref = 1/fref;
    Fs_per_cycle = 400;
    dt = Tref/Fs_per_cycle;
    t = 0:dt:(Ncyc_avg*Tref);
    vref_d = squareWave(fref, t, 0, 1) > 0.5;
    vfb_d = squareWave(ffb, t, theta, 1) > 0.5;
    N = length(t);
    UP = zeros(1,N); DN = zeros(1,N);
    up = 0; dn = 0;
    ref_prev = vref_d(1);
    fb_prev = vfb_d(1);
    for k = 2:N

```

```

    ref_rise = (~ref_prev) && vref_d(k);
    fb_rise = (~fb_prev) && vfb_d(k);
    if ref_rise, up = 1; end
    if fb_rise, dn = 1; end
    if up && dn % simultaneous ->
ideal instantaneous reset
        up = 0; dn = 0;
    end
    UP(k) = up; DN(k) = dn;
    ref_prev = vref_d(k);
    fb_prev = vfb_d(k);
end
ve = Vdd * (UP - DN);
idx0 = round(Nskip*Tref/dt) + 1;
avgVe = mean(ve(idx0:end));
end
function Kd = centeredSlope(theta_vec, Ve,
theta0)
    % Numerical derivative (central difference)
of Ve(theta) at theta0,
    % using the two grid points straddling
theta0.
    [~, idx] = min(abs(theta_vec - theta0));
    if idx == 1
        Kd = (Ve(idx+1) - Ve(idx)) /
(theta_vec(idx+1) - theta_vec(idx));
    elseif idx == length(theta_vec)
        Kd = (Ve(idx) - Ve(idx-1)) / (theta_vec(idx)
- theta_vec(idx-1));
    else
        Kd = (Ve(idx+1) - Ve(idx-1)) /
(theta_vec(idx+1) - theta_vec(idx-1));
    end
end
end

```